

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет Программной инженерии и компьютерной техники

Системы искусственного интеллекта
Лабораторная работа №3

Выполнил студент
Егорова Варвара Александровна
Группа Р3323
Преподаватель: Королева Юлия Александровна

Содержание	
Текст задания.....	3
Алгоритм.....	4
Реализация.....	5
Оценка модели.....	9
Вывод.....	11
Приложение.....	12

Текст задания

Реализовать класс модели SVM и обучить её на заданном датасете (bi2013a).

Алгоритм

Ниже представлены псевдокоды алгоритма модели SVM.

Для обучающей выборки $\{(x_i, y_i)\}_{i=1}^n$, $x_i \in R^d$, $y_i \in \{-1, +1\}$

ищем веса w и смещение b : $\min_{w, b, \xi} \frac{1}{2} ||w||^2 + C \sum_{i=1}^n \xi_i$ при

ограничениях $y_i(w^\top x_i + b) \leq 1 - \xi_i$, $\xi_i \geq 0$.

Реализация

Далее на листинге 1 представлена реализация класса SVM-модели.

Листинг 1 - Реализация SVM-модели.

```
import numpy as np
from sklearn.metrics import accuracy_score, f1_score
import matplotlib.pyplot as plt

class SVM:
    def __init__(self, C: float = 1.0, learning_rate: float = 0.1,
fit_intercept: bool = True, epochs: int = 10, batch_size: int = 50,
device: torch.device = device):
        self.C = C
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.batch_size = batch_size
        self.device = device
        self.weights = None
        self.fit_intercept = fit_intercept
        self.bias = 0.0 if fit_intercept else None
        self.train_loss_history = []
        self.train_accuracy_history = []
        self.val_loss_history = []
        self.val_accuracy_history = []
        self.is_fitted = False

    def _learn(self, X_train: torch.Tensor, y_train: torch.Tensor, X_val:
torch.Tensor, y_val: torch.Tensor):
        n_samples, n_features = X_train.shape
        params = [p for p in [self.weights, self.bias] if p is not None]
        optimizer = torch.optim.Adam(params, lr=self.learning_rate)

        for i in range(1, self.epochs + 1):
            sum_loss = 0.0
            sum_correct = 0
            n_seen = 0
            for start in range(0, n_samples, self.batch_size):
                end = start + self.batch_size
                X_batch = X_train[start:end].to(self.device)
                y_batch = y_train[start:end].to(self.device)

                scores = X_batch @ self.weights + (self.bias if
self.fit_intercept else 0.0)

                hinge = torch.clamp(1 - y_batch * scores, min=0)
                loss = 0.5 * torch.sum(self.weights ** 2) + self.C *
```

```

    hinge.mean()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    preds = torch.sign(scores)
    sum_correct += (preds == y_batch).sum().item()
    n_seen += y_batch.size(0)
    sum_loss += loss.item() * y_batch.size(0)

    epoch_loss = sum_loss / n_seen
    epoch_acc = sum_correct / n_seen

    self.train_loss_history.append(epoch_loss)
    self.train_accuracy_history.append(epoch_acc)

    with torch.no_grad():
        val_scores = X_val @ self.weights + (self.bias if
self.fit_intercept else 0.0)
        val_hinge = torch.clamp(1 - y_val * val_scores, min=0)
        val_loss = 0.5 * torch.sum(self.weights ** 2).item() + self.C *
val_hinge.mean().item()
        val_preds = torch.sign(val_scores)
        val_acc = (val_preds == y_val).float().mean().item()
        self.val_loss_history.append(val_loss)
        self.val_accuracy_history.append(val_acc)

    print(f"Epoch {i}/{self.epochs} -- Train Loss: {epoch_loss:.4f},
Train Acc: {epoch_acc:.4f} | Val Loss: {val_loss:.4f}, Val Acc:
{val_acc:.4f}")

    def fit(self, X_train: torch.Tensor, y_train: torch.Tensor, X_val:
torch.Tensor, y_val: torch.Tensor):
        n_samples, n_features = X_train.shape
        self.weights = torch.zeros(n_features, device=self.device,
requires_grad=True)

        if self.fit_intercept:
            self.bias = torch.tensor(0.0, device=self.device,
requires_grad=True)

        self._learn(X_train=X_train, y_train=y_train, X_val=X_val,
y_val=y_val)

```

```

self.is_fitted = True

def predict(self, X_test: torch.Tensor) -> torch.Tensor:
    if not(self.is_fitted):
        raise RuntimeError("Model is not fitted yet!")

    s = X_test @ self.weights + (self.bias if self.fit_intercept else
0.0)
    preds = torch.sign(s)

    return preds

def evaluate(self, y_pred: torch.Tensor, y_test: torch.Tensor):
    y_pred = y_pred.detach().cpu().numpy()
    y_test = y_test.detach().cpu().numpy()

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, pos_label=1)

    print(f"Accuracy: {acc:.4f} | F1: {f1:.4f}")

def plot_metrics(self):
    fig, axes = plt.subplots(1, 2, figsize=(15, 7))

    # Loss
    axes[0].plot(self.train_loss_history, label='Train', linewidth=2)
    axes[0].plot(self.val_loss_history, label='Validation', linewidth=2)
    axes[0].set_title("Loss")
    axes[0].set_xlabel("Epoch")
    axes[0].set_ylabel("Loss")
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # Accuracy
    axes[1].plot(self.train_accuracy_history, label='Train',
linewidth=2)
    axes[1].plot(self.val_accuracy_history, label='Validation',
linewidth=2)
    axes[1].set_title("Accuracy")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("Accuracy")
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    plt.suptitle(f"SVM model training and validation metrics",

```

```
fontsize=16)  
plt.tight_layout()  
plt.show()
```


Оценка модели

Далее на Листинге 2 представлен результат работы модели.

Листинг 2 - Результат работы модели

```
Epoch 10/100 -- Train Loss: 0.3375, Train Acc: 0.8324 | Val Loss: 0.3298, Val Acc: 0.8358
Epoch 20/100 -- Train Loss: 0.3378, Train Acc: 0.8325 | Val Loss: 0.3332, Val Acc: 0.8358
Epoch 30/100 -- Train Loss: 0.3385, Train Acc: 0.8325 | Val Loss: 0.3303, Val Acc: 0.8358
Epoch 40/100 -- Train Loss: 0.3380, Train Acc: 0.8325 | Val Loss: 0.3300, Val Acc: 0.8358
Epoch 50/100 -- Train Loss: 0.3380, Train Acc: 0.8325 | Val Loss: 0.3298, Val Acc: 0.8358
Epoch 60/100 -- Train Loss: 0.3379, Train Acc: 0.8325 | Val Loss: 0.3305, Val Acc: 0.8358
Epoch 70/100 -- Train Loss: 0.3382, Train Acc: 0.8324 | Val Loss: 0.3297, Val Acc: 0.8358
Epoch 80/100 -- Train Loss: 0.3382, Train Acc: 0.8325 | Val Loss: 0.3312, Val Acc: 0.8358
Epoch 90/100 -- Train Loss: 0.3379, Train Acc: 0.8325 | Val Loss: 0.3300, Val Acc: 0.8358
Epoch 100/100 -- Train Loss: 0.3380, Train Acc: 0.8325 | Val Loss: 0.3300, Val Acc: 0.8358
Accuracy: 0.8346
```

Далее на рисунке 1 представлен график изменения loss и accuracy по эпохам в ходе обучения модели.

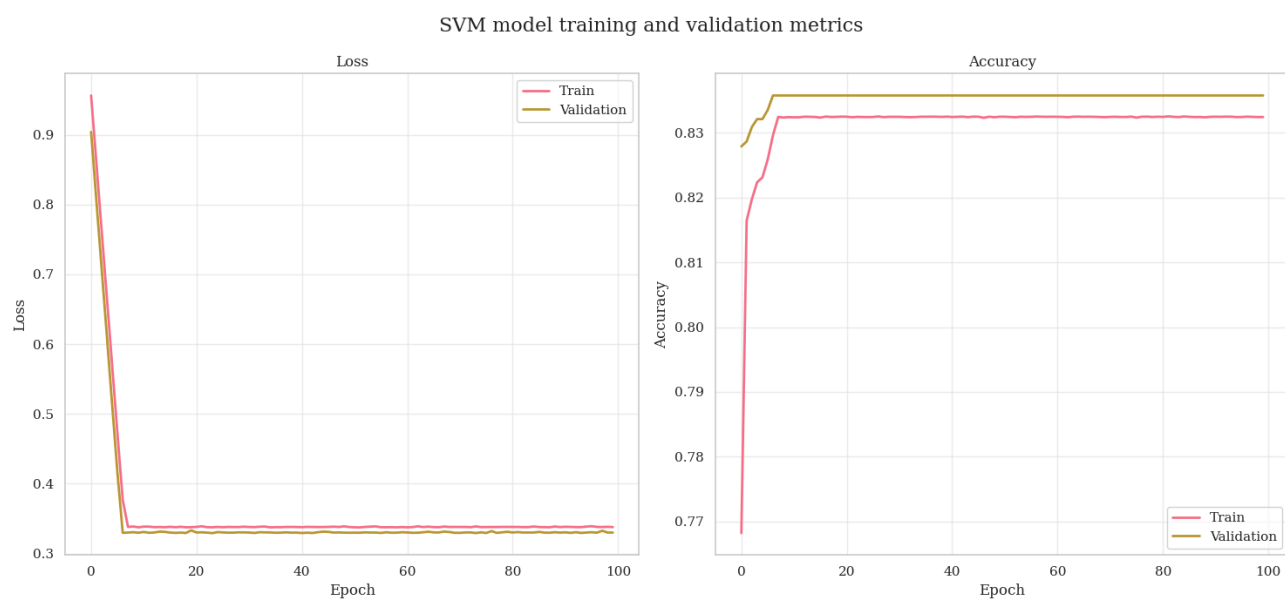


Рисунок 1 - Графики изменения Loss и Accuracy.

Вывод

В процессе выполнения лабораторной работы я познакомилась с устройством модели SVM, реализовала класс и обучила её на реальных данных.

Приложение

Исходный код лабораторной работы представлен по ссылке:
<https://colab.research.google.com/drive/13IxU4b4oQvIiRYoXMzTv845fuALgdG1q?usp=sharing>